

Лабораторная работа № 12

Итоговый инфраструктурный проект Linux-инфраструктуры

Продолжительность

4 занятия по 90 минут.

Дополнительно:

- самостоятельная подготовка документации;
- демонстрация проекта;
- выполнение отказных и восстановительных сценариев.

Формат работы

- индивидуально;
- в паре;
- небольшой командой — по решению преподавателя.

При командном выполнении для каждого сервиса назначается ответственный за:

- настройку;
 - диагностику;
 - восстановление;
 - объяснение работы сервиса на защите.
-

1. Цель работы

Объединить технологии модуля в связанную Linux-инфраструктуру и доказать её работоспособность через пользовательские, отказные и восстановительные сценарии.

В ходе проекта необходимо:

- спроектировать роли серверов;
- определить Naming Standard;
- подготовить Address Plan;
- разделить Production, Management, Backup и VPN Traffic;
- настроить внутренний DNS;
- организовать удалённый доступ через OpenVPN;
- развернуть два Web Backend;
- настроить балансировку HAProxy;
- подключить централизованный мониторинг Zabbix;
- настроить резервное копирование;

- выполнить Test Restore;
 - использовать Git для контроля конфигураций;
 - подключить Bash Health Check;
 - выполнить Failure Test;
 - выполнить Reboot Test;
 - подготовить исполнительную и эксплуатационную документацию.
-

2. Состав проекта

2.1. Обязательное ядро

В обязательную часть входят:

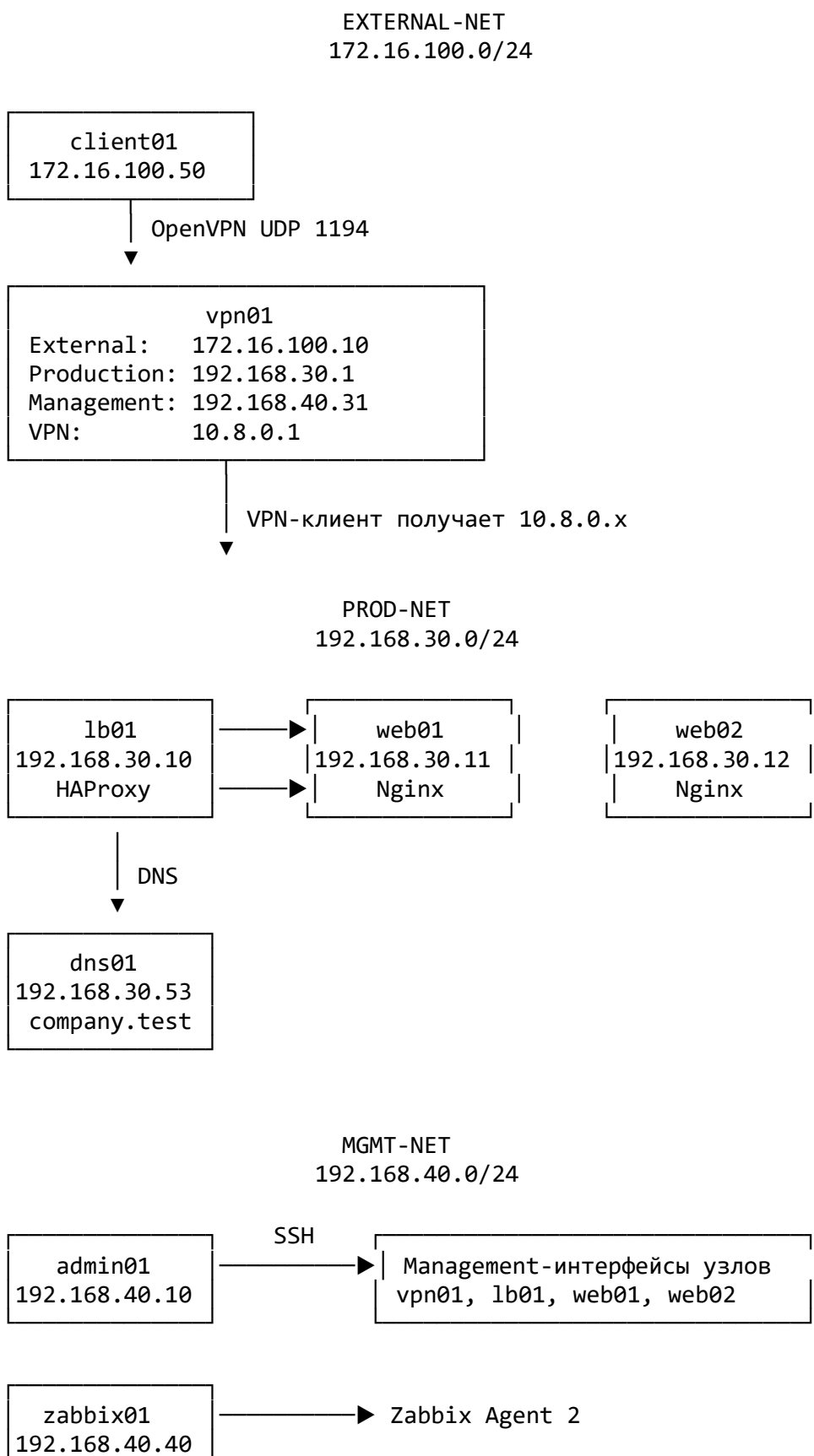
1. dns01 — внутренний DNS.
2. vpn01 — OpenVPN Server.
3. lb01 — HAProxy.
4. web01 — первый Nginx Backend.
5. web02 — второй Nginx Backend.
6. zabbix01 — Zabbix Server.
7. backup01 — BorgBackup Repository.
8. admin01 — административная рабочая станция.
9. client01 — внешний VPN-клиент.
10. Git-репозиторий инфраструктуры.
11. Bash-сценарий server-health.sh.

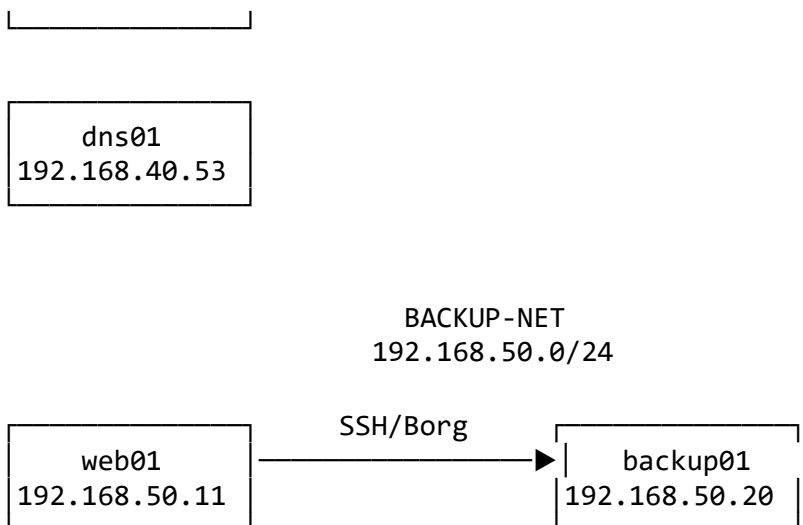
2.2. Расширенная часть

Дополнительно можно реализовать:

- Mailcow;
 - контейнерное приложение Docker;
 - отдельную Backup Network;
 - второй HAProxy и Keepalived;
 - автоматический Backup через systemd timer;
 - автоматическую проверку конфигураций;
 - централизованный Remote Git Repository;
 - TLS Termination на HAProxy;
 - Active Checks Zabbix;
 - автоматический Synthetic Test.
-

3. Итоговая архитектура





4. Сети VirtualBox

Название сети	Подсеть	Назначение
external-net	172.16.100.0/24	Внешний VPN-клиент
prod-net	192.168.30.0/24	Пользовательские сервисы
mgmt-net	192.168.40.0/24	SSH, Zabbix и управление
backup-net	192.168.50.0/24	Передача резервных копий
OpenVPN TUN	10.8.0.0/24	Адреса VPN-клиентов

В обязательном минимальном варианте Backup Traffic допускается передавать через Management Network. Это ограничение необходимо явно указать в документации.

5. Адресный план

5.1. Основные адреса

Узел	Production IP	Management IP	Backup IP
vpn01	192.168.30.1	192.168.40.31	—
lb01	192.168.30.10	192.168.40.32	—
web01	192.168.30.11	192.168.40.33	192.168.50.11
web02	192.168.30.12	192.168.40.34	192.168.50.12
zabbix01	—	192.168.40.40	—
backup01	—	192.168.40.50	192.168.50.20
dns01	192.168.30.53	192.168.40.53	—

Узел	Production IP	Management IP	Backup IP
admin01	—	192.168.40.10	—

5.2. Внешние адреса

Узел	Адрес
vpn01	172.16.100.10
client01	172.16.100.50

5.3. VPN

Объект	Адрес
OpenVPN Server	10.8.0.1
VPN Clients	10.8.0.0/24

5.4. Default Gateway

Для узлов Production Network:

192.168.30.1

Шлюзом является vpn01, обеспечивающий возврат трафика к VPN-клиентам.

Management и Backup Networks используются как непосредственно подключённые изолированные сети. IP Forwarding между ними не включается.

6. Naming Standard

Имена создаются по правилу:

<роль><номер>.company.test

Примеры:

vpn01.company.test
lb01.company.test
web01.company.test
web02.company.test
zabbix01.company.test
backup01.company.test
dns01.company.test

Сервисное имя Web-приложения:

web.company.test

Оно указывает на HAProxy:

web.company.test → 192.168.30.10

Требования

Для каждого сервера должны совпадать:

- Linux Hostname;
- FQDN;
- имя Host в Zabbix;
- имя в документации;
- имя в Git-каталогах и Runbook.

Проверка:

```
hostname  
hostname -f  
hostnamectl
```

7. Таблица разрешённых потоков

Источник	Назначение	Порт	Решение
client01	vpn01 External IP	UDP 1194	Разрешить
VPN Client	lb01 Production IP	TCP 443	Разрешить
VPN Client	web01/web02	TCP 22	Запретить
VPN Client	dns01	UDP/TCP 53	Разрешить
admin01	Management IP серверов	TCP 22	Разрешить
zabbix01	Zabbix Agents	TCP 10050	Разрешить
Agents	zabbix01	TCP 10051	При Active Checks
lb01	web01/web02	TCP 80	Разрешить
web01	backup01 Backup IP	TCP 22	Разрешить
Любой другой источник	Management UI	—	Запретить
Любой другой поток	Между зонами	—	Запретить

Firewall Matrix должна совпадать с фактическими правилами nftables, UFW или другого выбранного Firewall.

8. Исходное состояние

До начала работы:

- созданы виртуальные машины;
- установлена базовая Linux-система;

- добавлены сетевые адаптеры;
- выполнены Snapshots;
- студент имеет доступ к локальной консоли;
- нет опубликованных реальных секретов;
- сохранены результаты предыдущих лабораторных работ.

Разрешается повторно использовать VM, настроенные в лабораторных работах № 4–11.

9. Базовые задания

Задание 1. Подготовить проектную документацию

До установки и изменения сервисов создать каталог:

```
mkdir -p ~/linux-final-project/documentation
cd ~/linux-final-project
```

Создать документы:

```
documentation/
├── topology.md
├── address-plan.md
├── dns-zone-table.md
├── firewall-matrix.md
├── dependency-matrix.md
├── software-versions.md
├── change-record.md
└── acceptance-matrix.md
```

В topology.md указать

- список узлов;
- роли;
- сетевые адаптеры;
- сетевые зоны;
- пользовательский путь;
- административный путь;
- Backup Path.

В address-plan.md указать

- Hostname;
- FQDN;
- интерфейс;
- IP-адрес;
- маску;

- Gateway;
- DNS;
- назначение интерфейса.

B dependency-matrix.md ОПИСАТЬ

Сервис	Зависимости	Последствие отказа
OpenVPN	DNS, время, CA, Firewall	Клиент не получает Tunnel
HAProxy	web01, web02, сеть	Web недоступен или деградирован
Web Backend	Nginx, данные, диск	Ошибка приложения
Zabbix	Database, Agents, сеть	Нет актуального Monitoring
Backup	SSH, Repository, место	Нет точки восстановления
DNS	Сеть, BIND, Zone File	Имена перестают разрешаться

Проверка

Адреса и имена во всех документах должны совпадать.

Задание 2. Проверить базовую ОС

На каждом сервере выполнить:

```
hostnamectl
hostname -f
cat /etc/os-release
uname -r
timedatectl
ip -br addr
ip route
```

Проверить ресурсы:

```
uptime
free -h
df -hT
df -i
systemctl --failed
```

Создать Baseline:

```
sudo sh -c '
{
    echo "===== DATE ====="
    date

    echo "===== HOST ====="
    hostnamectl
    hostname -f
```

```

echo "===== OS ====="
cat /etc/os-release
uname -r

echo "===== TIME ====="
timedatectl

echo "===== NETWORK ====="
ip -br addr
ip route

echo "===== RESOURCES ====="
uptime
free -h
df -hT
df -i

echo "===== FAILED SERVICES ====="
systemctl --failed
} > /root/final-project-baseline.txt
,

```

Ожидаемый результат

- Hostname соответствует Naming Standard;
- время синхронизировано;
- адреса соответствуют Address Plan;
- нет неожиданных Failed Units;
- файловые системы не заполнены;
- Management-интерфейсы не имеют лишнего Default Route.

Задание 3. Настроить внутренний DNS

На dns01 установить BIND:

```

sudo apt update
sudo apt install -y bind9 bind9-utils dnsutils

```

Создать резервную копию:

```

sudo cp -a \
/etc/bind/named.conf.local \
/etc/bind/named.conf.local.backup

```

Открыть:

```

sudo nano /etc/bind/named.conf.local

```

Добавить:

```
zone "company.test" {  
    type master;  
    file "/etc/bind/db.company.test";  
};
```

Создать Zone File:

```
sudo nano /etc/bind/db.company.test
```

Добавить:

```
$TTL 3600
```

```
@    IN SOA dns01.company.test. admin.company.test. (  
        2026072801  
        3600  
        900  
        604800  
        3600  
)
```

```
IN NS  dns01.company.test.
```

```
dns01      IN A    192.168.30.53  
vpn01      IN A    192.168.30.1  
lb01       IN A    192.168.30.10  
web        IN A    192.168.30.10  
web01      IN A    192.168.30.11  
web02      IN A    192.168.30.12
```

```
zabbix01   IN A    192.168.40.40  
backup01   IN A    192.168.50.20  
admin01    IN A    192.168.40.10
```

Проверить конфигурацию:

```
sudo named-checkconf
```

Проверить Zone:

```
sudo named-checkzone \  
    company.test \  
    /etc/bind/db.company.test
```

Ожидается:

```
zone company.test/IN: loaded serial 2026072801  
OK
```

Перезапустить:

```
sudo systemctl restart bind9
```

Проверить:

```
systemctl status bind9 --no-pager
sudo ss -lnup | grep ':53'
sudo ss -lnpt | grep ':53'
```

С admin01 или другого узла:

```
dig @192.168.40.53 +short web.company.test
dig @192.168.40.53 +short lb01.company.test
dig @192.168.40.53 +short web01.company.test
dig @192.168.40.53 +short web02.company.test
```

Ожидается:

```
192.168.30.10
192.168.30.10
192.168.30.11
192.168.30.12
```

Проверить системное разрешение:

```
getent hosts web.company.test
```

Если используется systemd-resolved:

```
resolvectl query web.company.test
```

Задание 4. Создать инфраструктурный Git-репозиторий

На admin01:

```
mkdir -p ~/linux-infrastructure
cd ~/linux-infrastructure
git init -b main
```

Создать структуру:

```
mkdir -p \
  configs/dns \
  configs/openvpn \
  configs/haproxy \
  configs/nginx/web01 \
  configs/nginx/web02 \
  configs/zabbix \
  configs/backup \
  scripts \
  documentation \
  runbooks \
  change-records \
  examples
```

Создать .gitignore:

Secrets

.env

*.key

*.pem

*.p12

*.pfx

*.ovpn

secrets/

Backup files

*.bak

*.backup

*.dump

*.sql

Captures and logs

*.pcap

*.pcapng

*.log

Temporary files

tmp/

cache/

*~

*.swp

Добавить документацию проекта:

```
cp -a \  
  ~/linux-final-project/documentation/. \  
  documentation/
```

Перед первым Commit проверить потенциальные секреты:

```
grep -RIe \  
  --exclude-dir=.git \  
  'BEGIN .*PRIVATE KEY|password[[:space:]]*=|token[[:space:]]*=|api[_-]  
]?key[[:space:]]*=' \  
  . || true
```

Проверить:

```
git status
```

```
git diff
```

Добавить:

```
git add \  
  .gitignore \  
  documentation
```

Проверить Staging Area:

```
git diff --staged
```

Создать Commit:

```
git commit \  
-m "Initialize final Linux infrastructure project"
```

Правило проекта

Каждое значимое изменение выполняется по схеме:

```
Backup → Edit → Syntax Check → Functional Test  
→ git status → git diff → Secret Check  
→ git add → git diff --staged → Commit
```

Задание 5. Проверить OpenVPN

OpenVPN настраивается по результатам лабораторной работы № 4.

На vpn01 проверить:

```
systemctl status openvpn-server@server --no-pager  
sudo ss -ltnp | grep ':1194'  
ip -br addr show tun0  
sysctl net.ipv4.ip_forward
```

Ожидается:

```
active (running)  
10.8.0.1/24  
net.ipv4.ip_forward = 1
```

На client01 подключить валидный профиль.

После подключения:

```
ip -br addr show tun0  
ip route
```

Проверить маршрут:

```
ip route get 192.168.30.10
```

В выводе должен использоваться tun0.

Проверить DNS через VPN:

```
dig @192.168.30.53 \  
+short \  
web.company.test
```

Проверить разрешённый доступ:

```
curl -kI https://web.company.test/
```

Проверить запрещённый SSH:

```
nc -vz -w 5 192.168.30.11 22
nc -vz -w 5 192.168.30.12 22
```

Соединение не должно устанавливаться.

Проверить правила:

```
sudo nft list ruleset
```

Ожидаемая политика

VPN Client → web.company.test:443	ALLOW
VPN Client → web01/web02:22	DENY
VPN Client → dns01:53	ALLOW

Задание 6. Настроить два Web Backend

На web01 и web02 установить Nginx:

```
sudo apt update
sudo apt install -y nginx curl
```

На web01 создать страницу

```
sudo mkdir -p /srv/lab-web

sudo tee /srv/lab-web/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>web01</title>
</head>
<body>
    <h1>Response from web01</h1>
</body>
</html>
EOF
```

На web02 создать страницу

```
sudo mkdir -p /srv/lab-web

sudo tee /srv/lab-web/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>web02</title>
</head>
<body>
```

```
<h1>Response from web02</h1>
</body>
</html>
EOF
```

На каждом Backend создать конфигурацию Nginx:

```
server {
    listen 80 default_server;
    server_name _;

    root /srv/lab-web;
    index index.html;

    location = /health {
        access_log off;
        default_type text/plain;
        return 200 "OK\n";
    }

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Проверить:

```
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

Проверить напрямую с lb01:

```
curl http://192.168.30.11/
curl http://192.168.30.12/
curl -i http://192.168.30.11/health
curl -i http://192.168.30.12/health
```

Задание 7. Настроить HAProxy

На lb01 установить:

```
sudo apt update
sudo apt install -y haproxy socat curl
```

Создать Backup:

```
sudo cp -a \
    /etc/haproxy/haproxy.cfg \
    /etc/haproxy/haproxy.cfg.backup
```

Создать конфигурацию:

```

global
    log /dev/log local0
    log /dev/log local1 notice

    user haproxy
    group haproxy
    daemon

    stats socket /run/haproxy/admin.sock \
        mode 660 \
        level operator

defaults
    log global
    mode http

    option httplog
    option dontlognull

    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend fe_http
    bind 192.168.30.10:80

    http-request del-header X-Forwarded-For
    http-request add-header X-Forwarded-For %[src]

    default_backend be_web

backend be_web
    balance roundrobin

    option httpchk GET /health
    http-check expect status 200

    server web01 192.168.30.11:80 \
        check inter 2s fall 3 rise 2

    server web02 192.168.30.12:80 \
        check inter 2s fall 3 rise 2

```

Проверить:

```

sudo haproxy \
    -c \
    -f /etc/haproxy/haproxy.cfg

```

Ожидается:

Configuration file is valid

Применить:

```
sudo systemctl enable --now haproxy
sudo systemctl reload haproxy
```

Проверить:

```
systemctl status haproxy --no-pager
sudo ss -lntp | grep ':80'
```

Проверить распределение:

```
for i in $(seq 1 10); do
    curl \
        -s \
        -H 'Connection: close' \
        http://192.168.30.10/ |
        grep -o 'Response from web0[12]'
```

done

Оба Backend должны участвовать в обработке запросов.

Задание 8. Подключить Zabbix Monitoring

На наблюдаемых узлах установить Zabbix Agent 2:

- dns01;
- vpn01;
- lb01;
- web01;
- web02;
- backup01.

Основные параметры:

```
Server=192.168.40.40
ServerActive=192.168.40.40
Hostname=<точное имя узла>
```

Пример для web01:

```
Server=192.168.40.40
ServerActive=192.168.40.40
Hostname=web01
```

Проверить:

```
sudo systemctl enable --now zabbix-agent2
systemctl status zabbix-agent2 --no-pager
sudo ss -lntp | grep ':10050'
```

Во Frontend создать Hosts:

```
dns01
vpn01
lb01
web01
web02
backup01
```

Подключить Linux Template.

Обязательные проверки

Для всех узлов

- Agent Availability;
- CPU;
- Load;
- Available Memory;
- Filesystem Usage;
- Inode Usage;
- Network;
- Uptime.

Для vpn01

- OpenVPN Service;
- UDP 1194;
- интерфейс tun0.

Для lb01

- HAProxy Service;
- TCP 80;
- пользовательский HTTP-запрос;
- число Backend в состоянии UP.

Для web01 и web02

- Nginx Service;
- TCP 80;
- HTTP /health.

Для backup01

- возраст последнего успешного Backup;

- свободное место Repository;
- состояние Agent.

Обязательные Triggers

Условие	Severity
Agent unavailable	High
Nginx inactive	High
HAProxy inactive	Disaster
Один Backend DOWN	Average
Оба Backend DOWN	Disaster
Backup старше RPO	High
Filesystem заполнена	High

Проверить свежесть Latest Data. Отсутствие Problems не является доказательством исправности, если Items перестали обновляться.

Задание 9. Подключить Bash Health Check

На web01 разместить сценарий из лабораторной работы № 5:

```
/usr/local/sbin/server-health.sh
```

Проверить синтаксис:

```
sudo bash -n \  
/usr/local/sbin/server-health.sh
```

Проверить права:

```
sudo chown root:root \  
/usr/local/sbin/server-health.sh
```

```
sudo chmod 750 \  
/usr/local/sbin/server-health.sh
```

Запустить:

```
sudo /usr/local/sbin/server-health.sh \  
-s nginx \  
-w 80 \  
-c 90
```

Проверить код:

```
rc=$?  
echo "exit_code=$rc"
```

В штатном состоянии ожидается:

```
SUMMARY: OK
exit_code=0
```

Проверить журнал:

```
journalctl \
  -t server-health \
  -n 20 \
  --no-pager
```

Скопировать очищенную версию в Git:

```
cp \
  /usr/local/sbin/server-health.sh \
  ~/linux-infrastructure/scripts/
```

Проверить:

```
bash -n scripts/server-health.sh
shellcheck scripts/server-health.sh
git diff
```

Задание 10. Настроить Backup и выполнить Test Restore

Резервное копирование выполняется по схеме:

web01 → backup01

Используется BorgBackup, настроенный в лабораторной работе № 7.

Источники

```
/etc/nginx
/srv/lab-web
/usr/local/sbin/server-health.sh
```

На web01 настроить переменные текущей Root-сессии:

```
export BORG_REPO='ssh://backup-web01@192.168.50.20/srv/borg/web01'
export BORG_RSH='ssh -i /root/.ssh/id_ed25519_borg'
export BORG_PASSCOMMAND='cat /root/.config/borg/passphrase'
```

Перед Backup проверить источники:

```
findmnt /srv/lab-web || true
test -d /etc/nginx
test -f /srv/lab-web/index.html
test -x /usr/local/sbin/server-health.sh
```

Создать архив:

```
borg create \
  --stats \
  --show-rc \
  --compression lz4 \
  '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' \
  /etc/nginx \
  /srv/lab-web \
  /usr/local/sbin/server-health.sh
```

Проверить:

```
borg list
borg check \
  --verify-data \
  "$BORG_REPO"
```

Test Restore

Создать каталог:

```
sudo rm -rf /restore-test/web01
sudo mkdir -p /restore-test/web01
cd /restore-test/web01
```

Измерить время:

```
RESTORE_START=$(date +%s)
```

Восстановить последний архив:

```
LATEST_ARCHIVE=$(
  borg list \
    --short \
    --last 1
)

borg extract \
  "::${LATEST_ARCHIVE}"
```

Зафиксировать время:

```
RESTORE_END=$(date +%s)
RESTORE_SECONDS=$((RESTORE_END - RESTORE_START))
echo "restore_time_seconds=$RESTORE_SECONDS"
```

Проверить файлы:

```
find /restore-test/web01 \
  -type f \
  -printf '%p\n'
```

Проверить конфигурацию Nginx восстановленного стенда:

```
grep -R \  
  'server {' \  
  /restore-test/web01/etc/nginx
```

Проверить восстановленную страницу:

```
cat \  
  /restore-test/web01/srv/lab-web/index.html
```

Проверить восстановленный Bash-сценарий:

```
bash -n \  
  /restore-test/web01/usr/local/sbin/server-health.sh
```

Ожидаемый результат

- Repository проходит Integrity Check;
 - нужные данные присутствуют;
 - восстановление выполняется в отдельный каталог;
 - рабочая система не перезаписывается;
 - фактическое время восстановления зафиксировано.
-

Задание 11. Проверить Firewall Matrix

На каждом узле вывести активные правила:

```
sudo nft list ruleset
```

Проверить разрешённые сценарии.

C admin01

```
ssh admin@192.168.40.32  
ssh admin@192.168.40.33  
ssh admin@192.168.40.34
```

C zabbix01

```
zabbix_get \  
  -s 192.168.40.33 \  
  -k agent.ping
```

Ожидается:

1

C client01 через VPN

```
curl -kI https://web.company.test/
```

Запрещённый доступ

```
nc -vz -w 5 192.168.30.11 22  
nc -vz -w 5 192.168.30.12 22
```

SSH с VPN Client должен быть запрещён.

Backup Network

На web01:

```
nc -vz \  
  192.168.50.20 \  
  22
```

На узле, который не является Backup Source, доступ к Backup Repository должен быть запрещён.

Задание 12. Выполнить отказные тесты

Каждый отказ выполняется по схеме:

```
Preconditions  
→ Failure Action  
→ Expected Symptom  
→ Zabbix Problem  
→ Diagnosis  
→ Recovery  
→ Confirmation
```

Сценарий 1. Отказ web01

Перед тестом проверить:

```
curl http://192.168.30.11/  
curl http://192.168.30.12/
```

На lb01 проверить оба Backend:

```
echo 'show stat' |  
  sudo socat \  
    stdio \  
    /run/haproxy/admin.sock |  
  grep -E 'be_web,(web01|web02)'
```

Остановить только web01:

```
sudo systemctl stop nginx
```

Подождать:

```
sleep 7
```

С client01:

```
for i in $(seq 1 6); do
    curl \
        -s \
        -H 'Connection: close' \
        http://web.company.test/ |
        grep -o 'Response from web0[12]'
```

done

Ожидается:

Response from web02

Проверить Zabbix Problem.

Восстановить:

```
sudo systemctl start nginx
```

Подождать:

```
sleep 5
```

Повторить HTTP-тест и дождаться закрытия Zabbix Problem.

Сценарий 2. Устаревший Backup

На backup01 установить Marker двухдневной давности:

```
date \
    --date='2 days ago' \
    +%s |
    sudo tee \
    /var/lib/backup-monitor/web01.last_success
```

Проверить пользовательский Item:

```
sudo -u zabbix \
    /usr/local/sbin/backup-age-zabbix.sh
```

Значение должно превышать:

86400

Проверить Zabbix Problem.

Вернуть актуальное время:

```
date +%s |
    sudo tee \
    /var/lib/backup-monitor/web01.last_success
```

Дождаться Recovery.

Сценарий 3. Отозванный VPN-сертификат

Преподаватель предоставляет:

- действующий профиль;
- профиль с отозванным сертификатом;
- актуальный CRL на vpn01.

Действующий профиль должен:

- установить Tunnel;
- получить VPN IP;
- открыть web.company.test.

Отозванный профиль должен быть отклонён до получения доступа к внутренней сети.

На vpn01 проверить:

```
sudo journalctl \
-u openvpn-server@server \
-b \
--since '-10 min'
```

Успешный результат:

Valid certificate → Tunnel established
Revoked certificate → Connection rejected

Отозванный сертификат является не поломкой VPN, а корректной работой политики безопасности.

Сценарий 4. Test Restore

Удалить только тестовую копию файла:

```
sudo rm -f \
/restore-test/web01/srv/lab-web/index.html
```

Повторно извлечь файл из архива:

```
cd /restore-test/web01
borg extract \
":::${LATEST_ARCHIVE}" \
srv/lab-web/index.html
```

Проверить:

```
test -f \
/restore-test/web01/srv/lab-web/index.html

cat \
/restore-test/web01/srv/lab-web/index.html
```

Рабочий файл /srv/lab-web/index.html не изменяется.

10. Reboot Test

10.1. Перед перезагрузкой

Проверить автозапуск:

```
systemctl is-enabled nginx
systemctl is-enabled haproxy
systemctl is-enabled openvpn-server@server
systemctl is-enabled zabbix-agent2
systemctl is-enabled docker
```

Проверить постоянные настройки:

```
grep -R \
  'net.ipv4.ip_forward=1' \
  /etc/sysctl.conf \
  /etc/sysctl.d 2>/dev/null
```

```
sudo findmnt --verify
```

```
sudo nft list ruleset
```

10.2. Выполнить перезагрузку ключевых серверов

Последовательно перезагрузить:

1. dns01;
2. backup01;
3. zabbix01;
4. web01;
5. web02;
6. lb01;
7. vpn01.

Команда:

```
sudo reboot
```

10.3. После перезагрузки

Проверить:

```
hostname -f
timedatectl
ip -br addr
```

```
ip route
systemctl --failed
```

Проверить сервисы:

```
systemctl is-active bind9
systemctl is-active nginx
systemctl is-active haproxy
systemctl is-active openvpn-server@server
systemctl is-active zabbix-agent2
systemctl is-active docker
```

Проверить Mounts:

```
findmnt
```

Проверить Firewall:

```
sudo nft list ruleset
```

Проверить пользовательский сценарий:

```
curl -kI https://web.company.test/
```

Проверить VPN:

```
ip -br addr show tun0
```

Проверить Zabbix Latest Data.

Проверить Borg Repository:

```
borg list --last 3
```

Критерий

Проект не считается завершённым, если сервисы работают только до первой перезагрузки.

11. Документация проекта

11.1. As-Built Documentation

Документ должен описывать фактически развёрнутый стенд.

Включить:

- логическую схему;
- список VM;
- интерфейсы;
- Address Plan;
- DNS Zone Table;

- Firewall Matrix;
- Dependency Matrix;
- список открытых портов;
- конфигурационные файлы;
- версии ПО;
- Backup Schedule;
- Retention;
- Zabbix Hosts, Items и Triggers;
- известные ограничения.

11.2. Эксплуатационная инструкция

Для каждого сервиса указать:

1. Назначение.
2. Сервер и IP.
3. FQDN.
4. Зависимости.
5. Порты.
6. Конфигурационные файлы.
7. Команды ежедневной проверки.
8. Нормальный ожидаемый результат.
9. Журналы.
10. Типовые Alerts.
11. Безопасное изменение.
12. Rollback.

11.3. Recovery Runbooks

Минимум создать Runbooks:

```
runbooks/
├── dns-recovery.md
├── vpn-recovery.md
├── haproxy-web-recovery.md
├── zabbix-recovery.md
└── backup-restore.md
```

Каждый Runbook содержит:

- Preconditions;
- Failure Action;
- Symptoms;
- Diagnosis;
- Fix;

- Recovery;
- Confirmation;
- Escalation.

11.4. Change Record

Шаблон:

Поле	Значение
Дата	
Сервис	
Причина изменения	
Исполнитель	
Изменённые файлы	
Backup перед изменением	
Syntax Check	
Functional Test	
Git Commit	
Monitoring Result	
Rollback	
Итог	

12. Таблица конфигурационных файлов

Сервис	Основной файл	Проверка
DNS	/etc/bind/db.company.test	named-checkzone
OpenVPN	/etc/openvpn/server/server.conf	Подключение клиента
HAProxy	/etc/haproxy/haproxy.cfg	haproxy -c
Nginx	/etc/nginx/sites-enabled/	nginx -t
Zabbix Agent 2	/etc/zabbix/zabbix_agent2.conf	zabbix_agent2 -t
Docker	compose.yaml	docker compose config
Bash	/usr/local/sbin/server-health.sh	bash -n, ShellCheck
Backup	Backup Script и Environment File	Archive, Check, Restore

13. Инвентаризация версий

На каждом сервере зафиксировать:

```
cat /etc/os-release  
uname -r
```

По сервисам:

```
openvpn --version  
haproxy -vv  
nginx -v  
docker version  
docker compose version  
zabbix_agent2 -V  
borg --version  
git --version
```

Для полного варианта дополнительно:

- версия Mailcow;
 - версия Zabbix Server;
 - версия Database;
 - версия Docker Image;
 - Git Commit проекта.
-

14. Итоговая приёмочная матрица

14.1. Штатная работа

№	Проверка	Ожидаемый результат	Результат
1	DNS всех узлов	Адреса совпадают с планом	
2	Валидный VPN-профиль	Tunnel установлен	
3	DNS через VPN	web.company.test разрешается	
4	Web через VPN	HTTP/HTTPS работает	
5	Round Robin	Отвечают web01 и web02	
6	Health Check	Оба Backend UP	
7	Bash Health Check	Exit Code 0	
8	Zabbix Latest Data	Метрики актуальны	
9	Borg Archive	Архив создан	
10	Borg Check	Ошибок нет	
11	Test Restore	Данные восстановлены	

14.2. Безопасность

№	Проверка	Ожидаемый результат	Результат
1	VPN → Web	Разрешено	
2	VPN → SSH Backend	Запрещено	
3	admin01 → SSH	Разрешено	
4	Zabbix → Agent	Разрешено	
5	Неавторизованный узел → Backup	Запрещено	
6	Revoked Certificate	Tunnel не создаётся	
7	Secret Check	Реальных секретов нет	
8	Management UI	Недоступны из User Network	

14.3. Отказ и восстановление

№	Проверка	Ожидаемый результат	Результат
1	web01 остановлен	Web работает через web02	
2	web01 Down	Zabbix Problem открыт	
3	web01 восстановлен	Problem закрыт	
4	Backup старше RPO	Zabbix Problem открыт	
5	Marker обновлён	Recovery	
6	Test Restore	Файл восстановлен	
7	Reboot Test	Сервисы запускаются автоматически	
8	Persistent Data	Данные сохраняются	

15. Расширенные задания

Расширенное задание 1. Подключить Mailcow

Развернуть внутреннюю Mailcow-платформу:

mail.company.test

Создать:

user1@company.test
user2@company.test

Проверить:

- DNS A и MX;
- HTTPS;
- Submission 587;
- IMAPS 993;

- внутренний Mail Flow;
 - очередь;
 - запрет Open Relay;
 - отказ Dovecot;
 - восстановление Dovecot;
 - Backup и Test Restore.
-

Расширенное задание 2. Развернуть Docker-приложение

Создать Compose-проект с:

- Nginx;
- read-only Bind Mount;
- Healthcheck;
- Restart Policy;
- ротацией журналов;
- no-new-privileges;
- лимитом CPU;
- лимитом RAM.

Проверить:

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs
```

Пересоздать контейнер и подтвердить сохранение Persistent Data.

Расширенное задание 3. Настроить Keepalived

Добавить второй балансировщик:

1b02

Настроить Virtual IP:

192.168.30.9

Сервисное имя:

web.company.test → 192.168.30.9

Проверить:

- нормальную работу Active HAProxy;

- переход VIP на второй узел;
 - возврат VIP;
 - одинаковую конфигурацию HAProxy;
 - отказ процесса HAProxy;
 - ограничения VirtualBox для VRRP и Promiscuous Mode.
-

Расширенное задание 4. Автоматизировать Backup

Создать:

```
web01-backup.service  
web01-backup.timer
```

Проверить:

```
systemctl list-timers  
systemctl status web01-backup.service  
journalctl -u web01-backup.service
```

Скрипт должен:

- проверять наличие Sources;
 - использовать Lock;
 - создавать Archive;
 - применять Retention;
 - обновлять Success Marker только после успешного завершения;
 - возвращать ненулевой Exit Code при ошибке.
-

16. Условия непринятия проекта

Проект считается незавершённым при наличии хотя бы одного критического нарушения:

- отсутствует проверенный Test Restore;
- студент не может объяснить процедуру восстановления;
- реальные секреты опубликованы в Git, отчёте или скриншотах;
- Open Relay разрешён;
- VPN-клиент получает прямой SSH-доступ к Backend;
- Management UI доступен из пользовательской сети;
- Firewall Matrix не соответствует фактическим правилам;
- невозможно объяснить роль DNS;
- Address Plan не совпадает с фактическими адресами;
- сервисы не восстанавливаются после Reboot;
- Zabbix показывает устаревшие или Unsupported Items;

- Backup создаётся из отсутствующего или несмонтированного Source;
 - Git содержит непроверенную конфигурацию;
 - документация описывает не тот стенд, который реально настроен.
-

17. Требования к отчёту

В отчёте представить:

1. Название и цель проекта.
2. Логическую схему.
3. Схему сетевых интерфейсов.
4. Address Plan.
5. Naming Standard.
6. DNS Zone Table.
7. Firewall Matrix.
8. Dependency Matrix.
9. Список версий.
10. Список конфигурационных файлов.
11. Результаты штатных тестов.
12. Результаты тестов безопасности.
13. Результаты отказных тестов.
14. Результат Test Restore.
15. Фактический RTO.
16. Результат Reboot Test.
17. Zabbix Hosts, Items и Triggers.
18. Git History.
19. Change Record.
20. Recovery Runbooks.
21. Известные ограничения.
22. Итоговый вывод.

Не требуется добавлять скриншот каждой команды. Достаточно минимальных доказательств, подтверждающих настройку и результат проверки.

18. Запрещённые данные в отчёте

Не включать:

- пароли;
- Private Keys;

- VPN Client Profiles;
- Borg Passphrase;
- TLS Private Keys;
- API Tokens;
- Secret Macros;
- содержимое .env;
- закрытые ключи DKIM;
- реальные пользовательские письма;
- PCAP с Credentials;
- полный authorized_keys;
- Backup с пользовательскими данными.

Использовать:

REDACTED
CHANGE_ME
SECRET_STORED_SEPARATELY

19. Чек-лист выполнения

Архитектура

- ☐ Определены роли всех серверов.
- ☐ Подготовлен Naming Standard.
- ☐ Подготовлен Address Plan.
- ☐ Подготовлена схема.
- ☐ Подготовлена Dependency Matrix.
- ☐ Подготовлена Firewall Matrix.

Базовая ОС

- ☐ Hostname соответствует документации.
- ☐ Время синхронизировано.
- ☐ IP-адреса постоянны.
- ☐ Нет неожиданных Failed Units.
- ☐ Management Network не используется как Transit Network.

DNS

- ☐ Создана зона company.test.
- ☐ DNS Zone проходит проверку.
- ☐ Все обязательные имена разрешаются.

- ☐ web.company.test указывает на HAProxy.

VPN

- ☐ Валидный сертификат создаёт Tunnel.
- ☐ VPN Client получает адрес 10.8.0.x.
- ☐ Через VPN доступен Web.
- ☐ Прямой SSH к Backend запрещён.
- ☐ Отозванный сертификат отклоняется.

Web

- ☐ web01 работает.
- ☐ web02 работает.
- ☐ Страницы различаются.
- ☐ /health возвращает HTTP 200.
- ☐ HAProxy Config проходит проверку.
- ☐ Round Robin работает.
- ☐ Один Backend может быть исключён без полного отказа сервиса.

Monitoring

- ☐ Все Hosts созданы в Zabbix.
- ☐ Latest Data актуальны.
- ☐ Нет необъяснимых Unsupported Items.
- ☐ Созданы Triggers для сервисов.
- ☐ Различаются деградация и полный отказ.
- ☐ После Recovery Problems закрываются.

Backup

- ☐ Sources проверяются перед Backup.
- ☐ Архив создан.
- ☐ Repository проходит Check.
- ☐ Настроен Retention.
- ☐ Выполнен Test Restore.
- ☐ Зафиксирован фактический RTO.
- ☐ Success Marker обновляется только после успеха.

Git и Bash

- ☐ Создан Infrastructure Repository.
- ☐ Конфигурации очищены от секретов.
- ☐ Bash проходит bash -n.

- ☐ Bash проходит ShellCheck.
- ☐ Проверен Exit Code.
- ☐ Перед Commit используется `git diff --staged`.
- ☐ Commit Messages объясняют изменения.

Финальная проверка

- ☐ Выполнен Failure Test.
 - ☐ Выполнен Recovery.
 - ☐ Выполнен Reboot Test.
 - ☐ Документация соответствует стенду.
 - ☐ Runbooks проверены.
 - ☐ Реальные секреты отсутствуют.
-

20. Контрольные вопросы

1. Почему в итоговом проекте необходимо явно определить расположение DNS-сервера?
2. Чем Production Traffic отличается от Management и Backup Traffic?
3. Почему отсутствие Problems в Zabbix не всегда доказывает исправность?
4. Какие проверки выполняются перед Git Commit конфигурации?
5. Почему Backup без Test Restore не считается достаточным?
6. Как доказать, что VPN разрешает нужный доступ и запрещает лишний?
7. Почему отказ одного Web Backend не должен приводить к полному отказу?
8. Какие нарушения должны приводить к неприятию инфраструктурного проекта?